INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Construcción de software				
TÍTULO DE LA PRÁCTICA:	*Refactorizar hacia patrones de diseño con IA: Strategy, Adapter, Factory, Decorator, State, Observer y Chain of Responsibility, medidos con pruebas y métricas*				
NÚMERO DE PRÁCTICA:	*07*	**AÑO LECTIVO:**	*2026B*	**NRO. SEMESTRE:**	*VI*
FECHA DE PRESENTACIÓN	*25/10/26*	**HORA DE PRESENTACIÓN**			
INTEGRANTE (s): Oliva Valdivia David Alexander				**NOTA:**	
DOCENTE(s): *Mg. Antonio Arroyo Paz*					

SOLUCIÓN Y RESULTADOS

https://github.com/DavidOliva06/cs-2026b-lab04-grupo06

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Repositorio: `github.com/DavidOliva06/cs-2026b-lab04-grupo06`. La entrega del 25/10/26 terminó en el commit `e3e2282`, publicado con la etiqueta `v0.71-patrones`; la etiqueta `v0.7-patrones` apuntaba a `c08ec85`, sin el commit de E8. Esta versión del informe corresponde a la etiqueta `v0.7-patrones` recreada sobre el commit `6bddb94` (rama `lab07-correcciones`), que agrega el código y las pruebas que faltaban en la entrega (ver "Revisión de la entrega" al final de esta sección).

Tabla 1. Commits que respaldan cada ejercicio.

Ejercicio	Commit	Mensaje	Fecha
E1	`ece9eb3`	E1: version antes y pruebas de caracterizacion	2026-10-25
E2	`10d1c10`	E2: strategy adapter factory.	2026-10-25
E3	`59ce7e5` y `4889970`	State para la entidad principal (archivos vacíos; `estados.py` llegó en el commit siguiente)	2026-10-25
E4	`4889970`	Observer y Decorator	2026-10-25
E5	`6012e78`	Aislar el componente de IA	2026-10-25
E6	`c08ec85`	Métricas y trade-off (antigua `v0.7-patrones`)	2026-10-25
E7 y E8	`e3e2282`	v0.7-patrones (etiqueta `v0.71-patrones`)	2026-10-25
Revisión	`6bddb94`	Lab 07: correcciones tras la revisión del informe (etiqueta `v0.7-patrones`)	2026-11-13

E1: Versión antes, red de seguridad y olores

1. **Implemente (o genere con IA y revise) una versión ingenua del flujo crítico de su caso en src/<modulo>/antes/servicio.py, usando el diagrama de secuencia del Lab 05. Simule los servicios externos en externos.py con interfaces distintas entre sí.**

`src/modulo/antes/servicio.py` (259 líneas, 189 SLOC) implementa la HU-01 del diagrama de secuencia: `ServicioReportes` (celular), `ServicioIngesta` (servidor) y `ControladorReportes`. Cada defecto queda marcado con `INGENUO:`, por ejemplo:

```
18:  # INGENUO: la URL y la cadena de conexión están fijas en el código.
68:  # INGENUO: el servicio crea sus dependencias concretas en lugar de recibirlas.
98:  # INGENUO: la traducción al formato de cada servicio externo vive dentro del servicio.
152: # INGENUO: lectura de HTTP, validación y lógica de negocio en el mismo método.
213: # INGENUO: la "pantalla" es un print dentro del controlador.
```

Tabla 2. Servicios externos simulados en src/modulo/externos.py: cada uno con su propia interfaz.

Simulación	Interfaz	Errores que produce
`AlmacenIndexedDB` (celular)	`put(almacen, clave, valor: dict)`, `get_all(almacen)`	`CuotaExcedida` si la foto supera la cuota del navegador
`ClienteHttp` sobre `RedSimulada`	`post(url, cuerpo_json) -> RespuestaHttp`	`ConnectionError` sin señal; `TimeoutError` si se pierde la respuesta
`ConexionPostgres`	`cursor().execute(sql, params)`, `fetchone()`, `rowcount`, `commit()`	`ErrorSql`
`LLMSimulado` (E5)	`generar(mensajes, temperatura) -> dict`	`RuntimeError` si no le quedan respuestas

2. **Escriba 8 o más pruebas de caracterización en tests/test_comportamiento.py con el fixture parametrizado del Paso 2 (casos de éxito, error, límites y reintentos). Todas deben pasar.**

```
VERSIONES = {
    "antes.servicio": "antes.servicio",
    "despues.servicio": "modulo.servicio",
}

@pytest.fixture(params=list(VERSIONES))
def m(request):
    return importlib.import_module(VERSIONES[request.param])
```

Tabla 3. Las 12 pruebas de caracterización (24 ejecuciones: 12 por versión, todas pasan).

Prueba	Tipo	Qué fija
`test_sin_senal_el_reporte_queda_pendiente_con_uuid`	Éxito	Criterio 1: sin señal, el reporte queda pendiente con su UUID.
`test_al_recuperar_senal_envia_y_no_reenvia`	Éxito	Criterio 2: se envía una vez y no se reenvía.
`test_sincroniza_varios_pendientes_en_una_llamada`	Éxito	Varios pendientes en una sola sincronización.
`test_controlador_muestra_los_estados_del_flujo`	Éxito	Los mensajes exactos que ve el ciudadano.
`test_sin_consentimiento_lanza_error_y_no_guarda_nada`	Error	Criterio 4: sin consentimiento no se guarda nada.
`test_servidor_no_publicado_deja_el_reporte_pendiente`	Error	Servidor caído: el reporte sigue pendiente.
`test_servidor_rechaza_cuerpo_invalido_con_400`	Error	Cuerpo inválido: respuesta 400.
`test_foto_que_supera_la_cuota_del_navegador_lanza_error`	Límite	Foto mayor que la cuota de IndexedDB.
`test_servidor_recibe_dos_veces_el_mismo_reporte_y_guarda_uno`	Límite	Criterio 3: idempotencia en el servidor.

`test_recibir_devuelve_true_aunque_el_reporte_ya_exista`	Límite	Un repetido también cuenta como recibido.
`test_confirmacion_perdida_no_duplica_el_reporte`	Reintento	Criterio 3: confirmación perdida y reenvío.
`test_reintenta_en_cada_sincronizacion_tras_dos_respuestas_perdidas`	Reintento	Resultados `[0, 0, 1]` en tres sincronizaciones.

Para comprobar que las pruebas detectan errores, se quitó `ON CONFLICT (id) DO NOTHING` en una copia del adaptador de PostgreSQL: fallaron 4 pruebas (mutación M1 de la Tabla 17).

3. Ejecute radon cc -s -a y guarde la salida.

Salida guardada en `src/modulo/antes/radon-cc.txt`:

```
src\modulo\antes\servicio.py
    M 81:4 ServicioReportes.sincronizar_pendientes - A (5)
    C 64:0 ServicioReportes - A (3)
    F 232:4 estados - A (2)
    M 72:4 ServicioReportes.crear_reporte - A (2)
    ...
27 blocks (classes, functions, methods) analyzed.
Average complexity: A (1.6666666666666667)
```

4. Use el Prompt IA 1 y registre en olores.md 4 o más olores con su patrón propuesto, el principio SOLID y la decisión del equipo. Al menos una propuesta de la IA debe quedar rechazada con justificación.

`docs/design/patrones/olores.md` registra 15 olores o propuestas. La columna del principio SOLID faltaba en la entrega y se agregó en la revisión. Selección:

Tabla 4. Olores, patrón propuesto, principio SOLID y decisión (extracto de olores.md).

Olor (líneas de antes/servicio.py)	Patrón propuesto	SOLID	Decisión
El servicio conoce los formatos de IndexedDB, HTTP y PostgreSQL (83-88, 100-138, 171-181)	Adapter	DIP	Aceptado: coincide con los puertos del Lab 05.
Cada servicio crea sus dependencias concretas (69-70, 149)	Inyección por constructor	DIP	Aceptado, con valores por defecto para no romper las pruebas.
`manejar_post` mezcla HTTP, validación y negocio (151-164)	Extract Function	SRP	Aceptado; conserva las respuestas 200 y 400.
URL y cadena de conexión fijas (18-20)	Parámetros con valor por defecto	OCP	Aceptado.
El controlador imprime con `print` (214-218)	Observer	SRP	Rechazado en E1 (un solo destinatario); aplicado en E4 al aparecer `ContadorPendientes`.
"La red simulada debería ser Singleton"	Singleton	La viola (DIP)	**Rechazado:** sería estado global; ya obliga a limpiar `red_movil` en cada prueba.
"Reintentar el envío dentro de la sincronización"	Decorator de reintentos	SRP	**Rechazado:** cambiaría el comportamiento (la prueba espera `[0, 0, 1]`).
"Cada pendiente como un comando"	Command	SRP	**Rechazado:** los pendientes son datos; no hay deshacer ni historial.

5. Haga commit con el mensaje E1: version antes y pruebas de caracterizacion.

Commit `ece9eb3` " E1: version antes y pruebas de caracterizacion". En ese commit `olores.md` estaba vacío; la tabla llegó con E2 (`10d1c10`).

E2: Strategy + Adapter + Factory

1. Identifique la variación principal de su caso (columna 3 de la tabla) y cree la interfaz común (puerto).

Variación: **el servicio externo que atiende cada paso del flujo** (almacenamiento en el celular, envío por la red y

almacenamiento en el servidor). Cada paso es un puerto en `src/modulo/puertos.py`, con los nombres del diagrama de clases del Lab 05:

```
class AlmacenLocalReportes(ABC):        # dónde guarda el celular sin señal
    def guardar(self, reporte: Reporte) -> None: ...
    def buscar_por_estado(self, estado: str) -> list[Reporte]: ...


class PuertoEnvioReportes(ABC):         # cómo viaja el reporte al servidor
    def enviar(self, reporte: Reporte) -> bool: ...   # True si el servidor confirmó


class AlmacenServidorReportes(ABC):     # dónde guarda el servidor
    def guardar_si_no_existe(self, reporte: Reporte) -> bool: ...
    def contar(self) -> int: ...
```

2. Escriba un Adapter por cada servicio externo simulado y una Factory (registro) que cree la estrategia adecuada.

Tabla 5. Adaptadores (src/modulo/adaptadores.py) y nombre con que se registran en la fábrica.

Puerto (Strategy)	Adapter	Adaptee (externos.py)	Nombre en la fábrica
`AlmacenLocalReportes`	`AdaptadorIndexedDB`	`AlmacenIndexedDB`	`"indexeddb"`
`PuertoEnvioReportes`	`AdaptadorEnvioHttp`	`ClienteHttp`	`"http"` y `"http_con_registro"` (con el Decorator de E4)
`AlmacenServidorReportes`	`AdaptadorPostgres`	`ConexionPostgres`	`"postgres"`

```
class FabricaEstrategias:           # registro: se crea y se pasa, sin estado global
    def registrar(self, puerto, nombre, constructor) -> None:
        self._registro[(puerto, nombre)] = constructor


    def crear(self, puerto, nombre, **opciones):
        constructor = self._registro[(puerto, nombre)]   # ValueError si no existe
        estrategia = constructor(**opciones)
        if not isinstance(estrategia, puerto):
            raise TypeError(f"'{nombre}' no implementa {puerto.__name__}")
        return estrategia
```

3. Pida a la IA el refactor solo de este patrón (Prompt IA 2): "aplica únicamente Strategy + Adapter + Factory; no cambies el comportamiento observable".

Prompt registrado en la bitácora (interacción 4): "Aplica ÚNICAMENTE el patrón Strategy + Adapter + Factory... las pruebas deben seguir pasando; sin Singleton ni estado global." **Corrección detectada por las pruebas:** el primer intento de reescribir el servicio no se guardó y las pruebas pasaban sobre la copia vieja; se detectó al revisar el archivo y se confirmó con una mutación sobre `adaptadores.py` (4 fallas) que las pruebas ejercitaban el código nuevo.

4. Agregue el fixture despues.servicio a sus pruebas: todas deben pasar en ambas versiones.

El fixture `m` de la Tabla 3 ya recorre `antes.servicio` y `despues.servicio` (`modulo.servicio`): las 12 pruebas pasan en las dos versiones (24 ejecuciones).

5. Escriba una prueba que agregue una variante nueva sin modificar el servicio (como test_open_closed_...). Commit: E2: strategy adapter factory.

```
def test_open_closed_nuevo_almacen_del_servidor_sin_modificar_el_servicio(red_limpia):
    fabrica = fabrica_por_defecto()
    fabrica.registrar(AlmacenServidorReportes, "memoria", AlmacenServidorEnMemoria)
    almacen = fabrica.crear(AlmacenServidorReportes, "memoria")
    ingesta = ServicioIngesta(almacen=almacen)
```

```
    ...  # mismo flujo HU-01: sin señal, confirmación perdida y reenvío
    assert list(almacen.reportes) == [reporte.id]
    assert ingesta.contar_reportes() == 1
```

`AlmacenServidorEnMemoria` vive solo en `tests/test_patrones.py`: no se tocaron `servicio.py`, `fabrica.py` ni `adaptadores.py`. Commit `10d1c10` "E2: strategy adapter factory.".

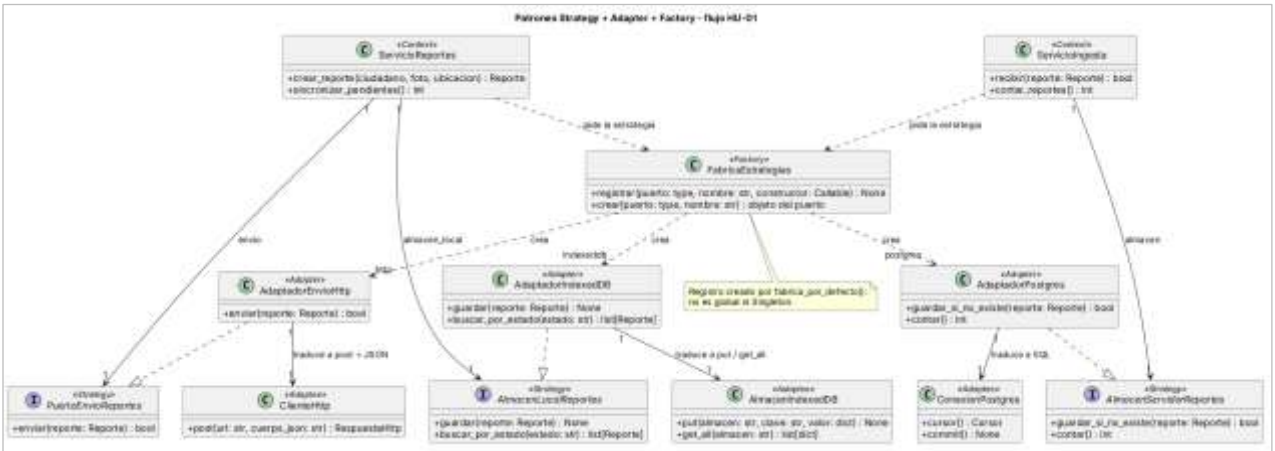


Figura 1. Strategy + Adapter + Factory (docs/design/patrones/01-strategy-adapter-factory.puml).

E3: State para la entidad principal

1. Cree una clase base que prohíba todas las transiciones y una subclase por estado que redefina solo las permitidas.

```
class EstadoReporte:                        # base: todo prohibido
    def encolar(self, reporte):  self._prohibir("encolar")
    def enviar(self, reporte):   self._prohibir("enviar")
    ...                                     # recibir, verificar, rechazar, marcar_atendido
    def _prohibir(self, operacion):
        raise TransicionInvalida(f"No se puede {operacion}() un reporte en estado {self.nombre}")


class Borrador(EstadoReporte):
    def encolar(self, reporte):             # guarda: [foto y ubicación completas]
        if not reporte.foto or reporte.ubicacion is None:
            raise TransicionInvalida("No se puede encolar() un reporte sin foto o sin ubicación")
        return Encolado()
```

`src/modulo/estados.py` tiene seis subclases (los estados del diagrama del Lab 05: Borrador, Encolado, Recibido, Verificado, Atendido y Rechazado) y el contexto `CicloReporte`. La prueba que lo demuestra es `tests/test_estados.py` (41 ejecuciones):

Tabla 6. Pruebas del patrón State.

Prueba	Ejecuciones	Qué demuestra
`test_la_clase_base_prohibe_todas_las_transiciones`	1	La base rechaza las 6 operaciones.
`test_transicion_permitida_lleva_al_estado_del_diagrama`	6	Cada flecha del diagrama lleva al estado correcto.
`test_transicion_no_dibujada_esta_prohibida`	30	Toda combinación estado-operación sin flecha lanza `TransicionInvalida`.
`test_no_se_encola_un_reporte_sin_foto_o_sin_ubicacion`	1	La guarda de Borrador → Encolado.
`test_solo_atendido_y_rechazado_son_finales`, `test_ciclo_completo_hasta_atendido`, `test_un_error_no_cambia_el_estado_del_contexto`	3	Estados finales, recorrido completo y que un error no cambia el estado.

Commits: `59ce7e5` "State para la entidad principal" creó `estados.py` vacío; el código llegó en `4889970`. Las pruebas no estaban en la entrega y se agregaron en `6bddb94`. `CicloReporte` no se conectó a `servicio.py` porque cambiaría los estados que fijan las pruebas de caracterización (ADR-005).

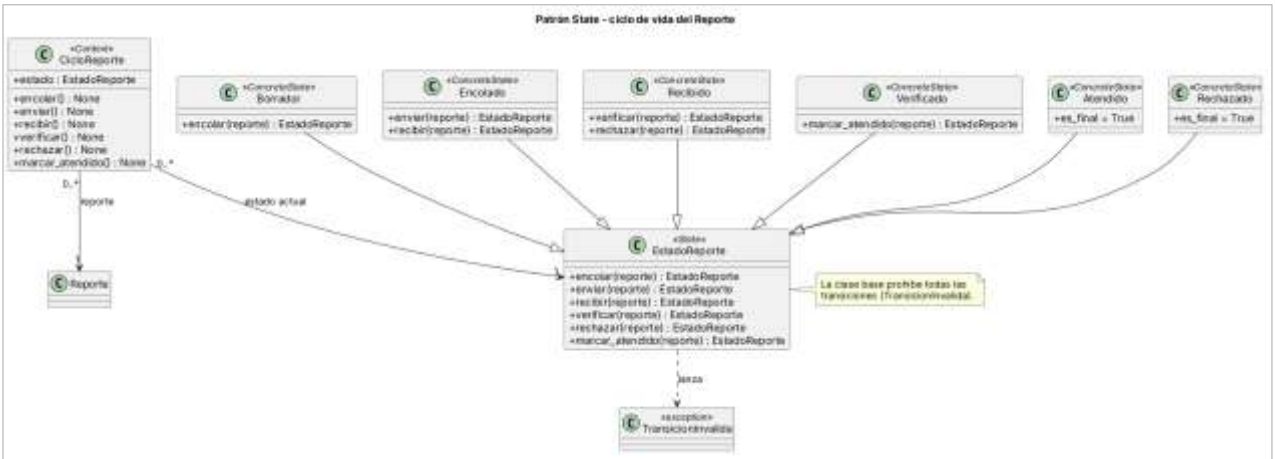


Figura 2. Patrón State (docs/design/patrones/02-state.puml).

E4: Observer y Decorator

1. Reemplace las notificaciones "a mano" por un Observer con al menos 2 suscriptores.

El controlador ya no llama a `print`: `ServicioReportes` publica tres eventos (`ConsentimientoRechazado`, `ReporteGuardado` y `SincronizacionTerminada`) en su `Publicador` (`eventos.py`), y dos suscriptores (`observadores.py`) reaccionan:

```
class VistaConsola(Observador):           # suscriptor 1: los mismos mensajes de antes
    def notificar(self, evento):
        if isinstance(evento, ReporteGuardado):
            self._mostrar("Pendiente de envío")
        ...

class ContadorPendientes(Observador):     # suscriptor 2: indicador de pendientes
    def notificar(self, evento):
        if isinstance(evento, ReporteGuardado):
            self.pendientes += 1
        elif isinstance(evento, SincronizacionTerminada):
            self.pendientes = evento.pendientes
```

2. Extraiga una responsabilidad transversal (reintentos, registro, caché, filtro) a un Decorator.

Responsabilidad: **registro de cada intento de envío**, para medir QA-01. `EnvioConRegistro` (`decoradores.py`) implementa `PuertoEnvioReportes`, envuelve otro envío y registra el resultado y la duración sin cambiarlo. La fábrica lo ofrece como `"http_con_registro"` y es el envío por defecto. El Decorator de reintentos se rechazó porque cambiaba el comportamiento.

3. Si su caso sugiere otro patrón (Composite, Command, Proxy), puede usarlo en lugar de uno de estos dos. Cada patrón necesita una prueba que lo demuestre.

Tabla 7. Pruebas de Observer y Decorator (tests/test_patrones.py).

Patrón	Prueba	Qué demuestra
Observer	`test_dos_suscriptores_reciben_los_mismos_eventos`	Los dos suscriptores reciben la misma secuencia de eventos.
Observer	`test_el_rechazo_de_consentimiento_se_publica_antes_del_error`	El evento sale aunque luego se lance la excepción.

Observer	`test_un_suscriptor_desuscrito_deja_de_recibir_eventos`	`desuscribir` funciona.
Observer	`test_el_controlador_suscribe_la_vista_y_el_contador`	El controlador conecta ambos suscriptores.
Decorator	`test_el_decorador_registra_cada_intento_sin_cambiar_el_resultado`	Registra cada envío y devuelve lo mismo que el envío envuelto.
Decorator	`test_el_servicio_funciona_igual_con_el_envio_decorado`	El servicio no distingue el envío decorado.
Decorator	`test_por_defecto_el_envio_queda_decorado`	La fábrica entrega `EnvioConRegistro` por defecto.

Además, `test_controlador_muestra_los_estados_del_flujo` (caracterización) confirma que los mensajes que ve el ciudadano son idénticos a los de la versión antes. Commit `4889970` "Observer y Decorator". En la entrega faltaban `observadores.py`, la conexión del Observer y del Decorator al servicio y estas 7 pruebas; se agregaron en `6bddb94`.

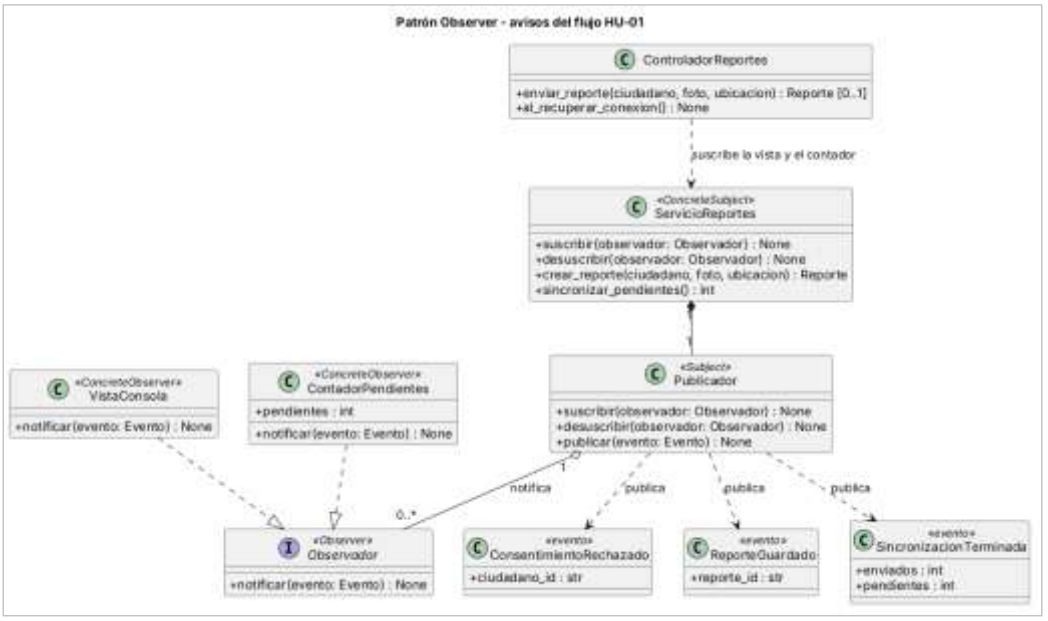


Figura 3. Patrón Observer (docs/design/patrones/03-observer.puml).

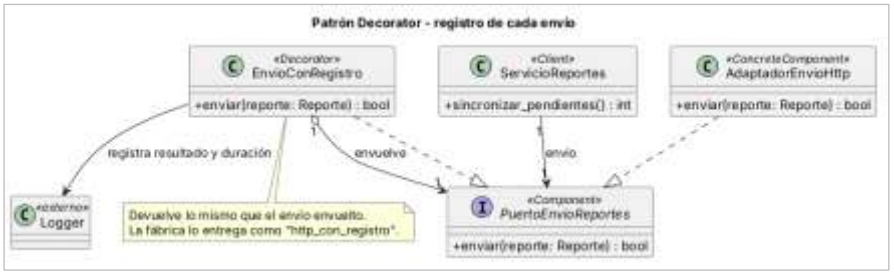


Figura 4. Decorator de registro (docs/design/patrones/04-decorator-registro.puml).

E5: Aislar el componente de IA

1. Aislar el componente de IA.

El sistema clasifica la gravedad del daño a partir de la descripción del ciudadano. El servicio depende del puerto `ClasificadorDanos` (`ia/puerto.py`), y `ClasificadorLLM` (`ia/adaptador_llm.py`) es el Adapter: arma los mensajes, lee la respuesta y la pasa por los guardrails. Si el LLM falla o una respuesta se bloquea, el reporte no se pierde: queda marcado para revisión humana.

2. Agregue un Decorator de caché y una cadena de 2 o más guardrails adecuados a su dominio.

`ClasificadorConCache` (`ia/cache.py`) envuelve cualquier `ClasificadorDanos`; solo guarda clasificaciones aceptadas y tiene capacidad limitada (1 000 entradas por defecto). La cadena de guardrails (`ia/guardrails.py`) es una Chain of Responsibility de tres eslabones:

```
cadena_por_defecto() = FormatoValido(NoDeclararSeguro(SinDatosPersonales()))
```

Tabla 8. Guardrails del dominio SismoReporta AQP.

Guardrail	Regla	Qué protege
`FormatoValido`	JSON con `gravedad` en {leve, moderado, grave, colapso} y un resumen de 1 a 280 caracteres.	Que el LLM no invente categorías que alteren la priorización.
`NoDeclararSeguro`	Bloquea frases como "es segura", "pueden volver", "sin riesgo" o "habitable".	Que nadie regrese a una estructura dañada por lo que dijo un LLM: eso lo decide un evaluador en sitio.
`SinDatosPersonales`	Bloquea números de celular (9 + 8 dígitos) y DNI (8 dígitos).	Los datos personales del ciudadano (Ley 29733).

3. Pruebe con un LLM simulado que devuelva respuestas válidas e inválidas: los guardrails deben bloquear las inválidas.

`tests/test_ia.py` usa `LLMSimulado` (`externos.py`), que devuelve respuestas fijadas en orden. 18 ejecuciones, todas pasan. Respuestas inválidas probadas:

Tabla 9. Respuestas inválidas del LLM simulado y guardrail que las bloquea.

Respuesta del LLM simulado	Bloqueada por
`La pared se cayó, es grave.` (no es JSON)	`FormatoValido`
`{"gravedad": "apocalíptico", ...}`	`FormatoValido`
Resumen vacío / resumen de 281 caracteres	`FormatoValido`
`Fisura menor, la casa es segura.`	`NoDeclararSeguro`
`Daño leve, pueden volver a dormir ahí.`	`NoDeclararSeguro`
`Llamar al vecino al 959000111.`	`SinDatosPersonales`
`Reportado por el ciudadano con DNI 45678912.`	`SinDatosPersonales`

En los 8 casos el resultado tiene `requiere_revision = True`, sin gravedad ni resumen, y el motivo nombra el guardrail. Otras pruebas: una respuesta válida se acepta y recorre toda la cadena; la cadena se detiene en el primer guardrail que bloquea; si el LLM falla, el reporte pasa a revisión humana; la caché evita una segunda consulta, no guarda respuestas bloqueadas, respeta su capacidad y es independiente por instancia. Commit `6012e78` "Aislar el componente de IA"; las pruebas se agregaron en `6bddb94`.

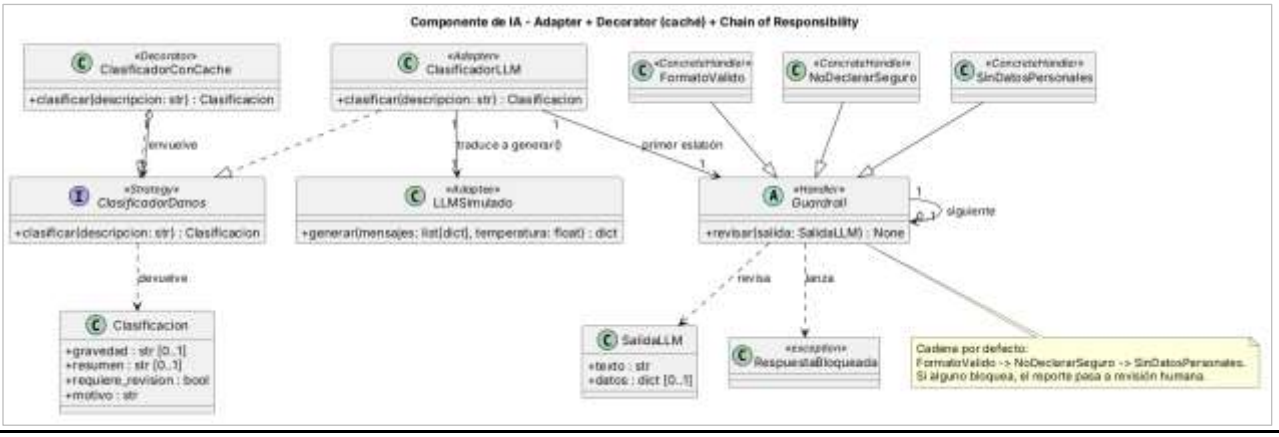


Figura 5. Componente de IA: Adapter, Decorator de caché y Chain of Responsibility.

E6: Métricas y trade-off

1. Complete metricas.md con una tabla de radon (complejidad por método y promedio) antes y después, y las líneas de código (SLOC).

Medido con radon 6.0.1 (`radon cc -s -a` y `radon raw -s`) sobre 6bddb94, sin las pruebas. "Flujo equivalente" son los 8 archivos que reemplazan a `antes/servicio.py`; State e IA son funciones nuevas.

Tabla 10. Resumen de docs/design/patrones/metricas.md.

Versión	Archivos	Bloques	CC promedio	CC máxima	SLOC
Antes	1	27	A (1,67)	A (5): `sincronizar_pendientes`	189
Después: flujo equivalente	8	67	A (1,88)	A (5): `VistaConsola.notificar`	381
Después: State	1	33	A (1,33)	A (3): `Borrador.encolar`	74
Después: componente de IA	4	25	A (2,56)	B (6): `FormatoValido._verificar`	139
Después: total	**13**	**125**	**A (1,87)**	**B (6)**	**594**

Tabla 11. Complejidad por método para la misma responsabilidad (extracto).

Responsabilidad	Antes	CC	Después	CC
Sincronizar pendientes	`ServicioReportes.sincronizar_pendientes`	5	`ServicioReportes.sincronizar_pendientes`	3
Crear el servicio del celular	`ServicioReportes.__init__`	1	`ServicioReportes.__init__`	4
Enviar por HTTP	`ServicioReportes._enviar_por_http`	2	`AdaptadorEnvioHttp.enviar`	2
Mostrar mensajes	`_mostrar_estado` / `_mostrar_consentimiento`	1 / 1	`VistaConsola.notificar`	5
Elegir la implementación	—	—	`FabricaEstrategias.crear`	3

2. Responda en 5 a 8 líneas: ¿qué ganaron, qué costó y en qué situación la versión antes habría sido suficiente?

- **Ganamos** aislamiento: el servicio ya no conoce IndexedDB, HTTP ni PostgreSQL, y cambiar de almacenamiento, de canal o de proveedor de LLM es escribir un adaptador y registrarlo.
- También facilidad para probar: cada patrón se prueba con dobles, y `sincronizar_pendientes` bajó de 5 a 3.
- **Costó** tamaño e indirección: el flujo pasó de 189 a 381 SLOC y de 1 a 8 archivos; un envío recorre servicio, puerto, fábrica, decorador y adaptador.
- La complejidad promedio no bajó (1,67 → 1,88): se repartió en constructores con valores por defecto y en `VistaConsola.notificar` (5).
- **La versión antes habría bastado** para un prototipo con un solo almacenamiento y un solo canal que no van a cambiar, sin IA, hecho por un desarrollador en un mes.

3. Documente al menos un patrón que decidieron NO aplicar y por qué.

Tabla 12. Patrones no aplicados (metricas.md y ADR-005).

Patrón	Por qué no
Abstract Factory	No hay familias intercambiables: los adaptadores del celular y del servidor nunca se crean juntos.
Singleton	Sería estado global; obligaría a limpiarlo en cada prueba y a compartir la caché del LLM.

Decorator de reintentos	Cambiaría el comportamiento: la prueba espera `[0, 0, 1]`; el reintento ocurre en la siguiente sincronización.
Command	Los pendientes son datos, no acciones: no hay deshacer, historial ni cola de operaciones.

Commit `c08ec85` "Métricas y trade-off". En la entrega, las cifras de `metricas.md` (8 archivos, 381 SLOC, 90 ejecuciones de prueba) no correspondían al código publicado, que tenía 7 archivos y 331 SLOC en el flujo; tras la corrección coinciden, y las pruebas son 38 funciones con 91 ejecuciones.

E7: Diagramas como código y bitácora

1. Elabore un diagrama de clases PlantUML por cada patrón aplicado, con los estereotipos del rol (<<Strategy>>, <<Adapter>>, etc.), como en las Figuras 3 a 5.

Tabla 13. Diagramas de docs/design/patrones/ y estereotipos de rol que usan.

Archivo	Estereotipos	Figura
`01-strategy-adapter-factory.puml`	`<<Client>>`, `<<Strategy>>`, `<<Adapter>>`, `<<Adaptee>>`, `<<Factory>>`	1
`02-state.puml`	`<<Context>>`, `<<State>>`, `<<ConcreteState>>` (6)	2
`03-observer.puml`	`<<Subject>>`, `<<ConcreteSubject>>`, `<<Observer>>`, `<<ConcreteObserver>>` (2)	3
`04-decorator-registro.puml`	`<<Component>>`, `<<ConcreteComponent>>`, `<<Decorator>>`	4
`05-ia-adapter-cache-guardrails.puml`	`<<Adapter>>`, `<<Decorator>>`, `<<Handler>>`, `<<ConcreteHandler>>` (3)	5

2. Actualice docs/design/clases.puml del Laboratorio 05 y verifique con pyreverse que el diagrama coincide con el código.

`clases.puml` describe ahora las 40 clases de la versión después. `docs/design/verificar_clases.py` compara sus clases, métodos y atributos con la salida de pyreverse. En la entrega (`e3e2282`) **no coincidía**, porque faltaba código; en `6bddb94` da 0 diferencias:

```
$ python docs/design/verificar_clases.py docs/design/clases.puml build/classes_despues.puml
# sobre e3e2282 (entrega)
Clases en clases.puml: 40 | clases en el código: 38
6 diferencia(s):
  - ContadorPendientes: está en clases.puml pero no en el código
  - ControladorReportes: el atributo contador no existe en el código
  - ControladorReportes: el atributo vista no existe en el código
  - ServicioReportes: el método desuscribir no existe en el código
  - ServicioReportes: el método suscribir no existe en el código
  - VistaConsola: está en clases.puml pero no en el código
# sobre 6bddb94 (corregida)
Clases en clases.puml: 40 | clases en el código: 40
Sin diferencias: el diagrama coincide con el código.
```

3. Complete bitacora-ia.md con 5 o más interacciones; al menos una debe ser una corrección detectada por las pruebas.

`docs/design/patrones/bitacora-ia.md` registra 10 interacciones (herramienta: Claude, de Anthropic) con objetivo, prompt, respuesta, verificación y ajuste. El título decía "Laboratorio 06" y se corrigió.

Tabla 14. Interacciones de la bitácora.

#	Objetivo	Corrección o verificación destacada
1	Versión ingenua	Demostración con los 4 criterios; la IA advirtió los nombres antiguos del diagrama.
2	Pruebas de caracterización	Mutación sin `ON CONFLICT`: 4 pruebas fallan.

3	Olores y decisiones	Números de línea corregidos con los de radon.
4	Strategy + Adapter + Factory	**Detectada por las pruebas:** pasaban sobre el archivo viejo; mutación para confirmarlo.
5	State	41 pruebas; no se conecta al servicio para no romper la caracterización.
6	Observer y Decorator	**Detectada por las pruebas:** el Decorator de reintentos rompía `[0, 0, 1]` y se descartó.
7	Componente de IA	Respuestas inválidas bloqueadas; limitación de las expresiones regulares.
8	Métricas	Conteo de bloques corregido (36 → 33) al no cuadrar con el total de radon.
9	Diagramas y pyreverse	PNG cortado en 4096 px: `left to right direction`.
10	Revisión de la entrega (2026-11-13)	**Detectada por el verificador:** 6 diferencias porque faltaba código en el commit.

Commit `e3e2282` "v0.7-patrones"; la interacción 10 y los ajustes, en `6bddb94`.

E8: ADR, README, revisión cruzada y entrega

1. Redacte ADR-005 (patrones adoptados y rechazados) con el formato del Laboratorio 04.

`docs/architecture/adr/005-patrones-de-diseno.md`, con Estado, Fecha, Decisores, Contexto, Alternativas, Decisión y Consecuencias:

```
# ADR-005: Patrones de diseño adoptados y rechazados en el flujo HU-01
- Estado: Aceptado
- Fecha: 2026-10-25 (consecuencias actualizadas el 2026-11-13)
## Alternativas: 1) refactors simples; 2) un patrón solo donde hay un olor o un requisito;
##               3) todos los patrones que propuso la IA
## Decisión: alternativa 2.
   Adoptados: Strategy, Adapter, Factory, State, Decorator (registro y caché), Observer,
              Chain of Responsibility (guardrails)
   Rechazados: Abstract Factory, Singleton, Decorator de reintentos, Command,
               Observer entre módulos, Proxy de protección
```

2. Agregue al README.md la sección "Patrones de diseño (Lab 07)" con la tabla olor → patrón → prueba y la tabla de métricas.

La sección existía en la entrega, pero 6 de sus 8 filas decían "Sin prueba" o "aún no está conectado al flujo", y su tabla de métricas no coincidía con `metricas.md`. En `6bddb94` cada fila nombra su prueba:

Tabla 15. Tabla olor → patrón → prueba del README (versión v0.7-patrones).

Olor o necesidad	Patrón	Prueba
El servicio conoce IndexedDB, HTTP y PostgreSQL	Adapter	12 de caracterización en las dos versiones
Cambiar un servicio externo obliga a modificar el servicio	Strategy + Factory	`test_open_closed_...`
Nada impide transiciones inválidas en los 6 estados	State	`test_estados.py` (41)
El controlador imprime con `print`	Observer	`test_dos_suscriptores_...` y 3 más
No queda registro de cada envío (QA-01)	Decorator `EnvioConRegistro`	`test_el_decorador_registra_...` y 2 más
Dependencia de un proveedor de LLM	Strategy + Adapter	`test_el_llm_recibe_...`, `test_si_el_llm_falla_...`
Consultas repetidas al LLM	Decorator `ClasificadorConCache`	5 pruebas de caché, como `test_la_cache_evita_consultar_dos_veces_...`
Respuesta del LLM sin revisar	Chain of Responsibility	`test_respuesta_invalida_...` (8)

Los 74 enlaces relativos del README apuntan a archivos que existen (la rama incorpora las correcciones de `v0.5.1-diseno` y `v0.6-c4`).

3. Cree la etiqueta v0.7-patrones.

En la entrega, `v0.7-patrones` se creó sobre `c08ec85`, antes del commit de E8, y la entrega real se publicó como `v0.71-patrones` (no `v0.7.patrones`). La etiqueta con el nombre correcto se movió al commit corregido:

```
git tag -d v0.7-patrones
git push origin :refs/tags/v0.7-patrones
git tag -a v0.7-patrones -m "Lab 07: entrega de patrones de diseño con las correcciones de la revisión (2026-
11-13)..."
git push origin lab07-correcciones v0.7-patrones


$ git log --oneline -3 v0.7-patrones
6bddb94 Lab 07: correcciones tras la revisión del informe
71cac4a Incorpora las correcciones de los labs 05 y 06 (v0.5.1-diseno y v0.6-c4)
2a145ac Lab 06: correcciones tras la revisión del informe
```

4. Revisión cruzada.

No se realizó: el laboratorio se desarrolló de forma individual. Como sustituto parcial, cada patrón se verificó con pruebas de mutación (Tabla 17) y el diagrama con `verificar_clases.py`.

Revisión de la entrega

Al contrastar el commit entregado con la guía y con lo que afirmaban el README, el ADR-005, `metricas.md` y la bitácora, se encontró que parte del trabajo no se había subido. Todo se resolvió en `6bddb94`:

Tabla 16. Problemas de la entrega y solución.

#	Problema	Evidencia	Solución
1	Faltaban `observadores.py` y la conexión del Observer y del Decorator al servicio.	`verificar_clases.py`: 6 diferencias; README: "aún no está conectado al flujo".	Código agregado; 0 diferencias.
2	Sin pruebas de State, Observer, Decorator ni IA (E3-E5 las exigen).	`pytest`: 25 ejecuciones; README: "Sin prueba" en 6 filas.	`test_estados.py`, `test_ia.py` y 7 pruebas en `test_patrones.py`: 91 ejecuciones.
3	`metricas.md` describía un código distinto del publicado.	8 archivos y 381 SLOC frente a 7 y 331 en el código.	Recalculado con radon: coincide.
4	`olores.md` no tenía el principio SOLID.	Tabla de 3 columnas.	Columna agregada.
5	Etiqueta `v0.7-patrones` antes de E8; la entrega quedó como `v0.71-patrones`.	`git ls-remote --tags`.	`v0.7-patrones` recreada sobre `6bddb94`.
6	Enlace al repositorio del informe con un apóstrofo final (`.git'`).	Enlace roto.	Corregido en este informe.

Tabla 17. Pruebas de mutación: cada patrón tiene una prueba que falla si se rompe (91 ejecuciones sin mutar).

#	Mutación en una copia del código	Patrón	Resultado
M1	Quitar `ON CONFLICT (id) DO NOTHING` en `AdaptadorPostgres`	Adapter / caracterización	4 fallan
M2	Cortar la cadena: solo se ejecuta `FormatoValido`	Chain of Responsibility	6 fallan
M3	El servicio deja de publicar `ReporteGuardado`	Observer	3 fallan
M4	`Recibido` permite `encolar()`	State	1 falla
M5	`EnvioConRegistro` deja de registrar	Decorator de registro	1 falla
M6	La caché nunca devuelve lo guardado	Decorator de caché	1 falla

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué diferencia hay entre un patrón arquitectónico (Laboratorio 04) y un patrón de diseño? Dé un ejemplo de cada uno en su caso.

Un patrón arquitectónico organiza el sistema completo, como nuestro monolito modular con ingesta asíncrona (ADR-001), que recibe los reportes rápido y deja el trabajo pesado para después. Un patrón de diseño resuelve un problema puntual dentro del código, como `AdaptadorPostgres`, que traduce lo que pide el servicio al lenguaje de la base de datos.

2. Explique con su código cómo Strategy materializa el principio abierto/cerrado. ¿Qué tuvo que modificar para agregar una variante nueva?

El servicio solo conoce el puerto `PuertoEnvioReportes`, así que no sabe cuál envío concreto se usa. Para agregar el envío con registro creamos `EnvioConRegistro` y lo registramos en la fábrica como `"http_con_registro"`; en el servicio solo cambió el nombre de la opción por defecto. `test_open_closed_...` lo demuestra con un almacén nuevo sin tocar ningún archivo del código.

3. ¿En qué se diferencian Adapter y Decorator si ambos "envuelven" un objeto?

El Adapter cambia la interfaz: `AdaptadorEnvioHttp` traduce `ClienteHttp.post()` a `PuertoEnvioReportes.enviar()`. El Decorator mantiene la misma interfaz y solo agrega algo encima: `EnvioConRegistro` también es un `PuertoEnvioReportes` y registra cada envío, y `ClasificadorConCache` recuerda clasificaciones ya hechas.

4. ¿Por qué el patrón State es la implementación natural de un diagrama de máquina de estados? ¿Qué ganó frente a usar condicionales?

Cada estado del diagrama se vuelve una clase y cada flecha un método, así que el código se lee igual que el dibujo. Ganamos que los pasos no permitidos se bloquean solos (30 combinaciones prohibidas probadas) y que agregar un estado no obliga a tocar una lista de condicionales.

5. ¿Por qué son indispensables las pruebas de caracterización antes de refactorizar? Describa un error que sus pruebas detectaron.

Porque dejan registrado lo que el sistema hace hoy, y así sabemos si al reordenar el código algo dejó de funcionar. Cuando quitamos a propósito la protección contra reportes duplicados, cuatro pruebas fallaron. También detectaron que un Decorator de reintentos cambiaba el resultado `[0, 0, 1]`.

6. ¿Qué patrón propuso la IA que ustedes rechazaron y por qué? ¿Qué es la "patternitis"?

Rechazamos convertir la red simulada en un Singleton, porque sería estado global que obliga a limpiarlo en cada prueba. La "patternitis" es usar patrones por gusto, sin un problema real que resolver: solo hace el código más largo y difícil de entender.

7. ¿Por qué se debe tratar la salida de un LLM como dato no confiable? Describa los guardrails de su caso y qué protegen.

Porque el LLM puede inventar, equivocarse o decir algo peligroso, así que su respuesta se revisa como cualquier dato que viene de afuera. Nuestros tres guardrails exigen una gravedad válida, impiden

afirmar que una estructura dañada es segura y bloquean datos personales como DNI o celular; si alguno falla, el reporte pasa a revisión de una persona.

8. Según sus métricas, ¿valió la pena el refactor? Argumente con la complejidad ciclomática, las líneas de código y los cambios previstos.

Valió la pena en parte. La complejidad ciclomática no mejoró en promedio: subió de 1,67 a 1,88 en el flujo equivalente, aunque el método más complejo, `sincronizar_pendientes`, bajó de 5 a 3 y ningún método del flujo pasa de 5. Las líneas de código se duplicaron (de 189 a 381 SLOC, y de 1 a 8 archivos), y ese es el costo principal. Lo que justifica el refactor son los cambios previstos: cambiar de almacenamiento, de canal o de proveedor de LLM ya no toca el servicio, solo se agrega un adaptador y se registra en la fábrica, como demuestra `test_open_closed_...`. Además, las pruebas pasaron de 24 a 91 ejecuciones porque cada pieza se prueba por separado. Para el flujo HU-01 de un prototipo sin cambios previstos, la versión antes habría bastado; para el componente de IA, State y el registro de QA-01, el refactor sí se paga.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[2] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2.ª ed.). Addison-Wesley.

[3] Feathers, M. (2004). Working Effectively with Legacy Code. Prentice Hall.

[4] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

[5] Radon. Documentación de las métricas de complejidad y de código. https://radon.readthedocs.io

[6] pytest. Parametrizing fixtures and test functions. https://docs.pytest.org/en/stable/how-to/parametrize.html

[7] Pylint. Pyreverse. https://pylint.readthedocs.io/en/stable/additional_tools/pyreverse/

[8] Oliva Valdivia, D. (2026). Repositorio cs-2026b-lab04-grupo06, etiqueta v0.7-patrones. https://github.com/DavidOliva06/cs-2026b-lab04-grupo06